

Filament Allocator

从Engine中我们可以看到HeapAllocatorArena和LinearAllocatorArena的声明和使用:

```
class FEngine : public Engine {
    ...
    LinearAllocatorArena mPerRenderPassAllocator;
    HeapAllocatorArena mHeapAllocator;
    ...
};
```

我们以HeapAllocatorArena来说明是如何使用的。

那HeapAllocatorArena和LinearAllocatorArena是什么呢?

```
using HeapAllocatorArena = utils::Arena<
    utils::HeapAllocator,
    utils::LockingPolicy::NoLock,
    utils::TrackingPolicy::Untracked,
    utils::AreaPolicy::NullArea>;

using LinearAllocatorArena = utils::Arena<
    utils::LinearAllocator,
    utils::LockingPolicy::NoLock>;
```

我们看到, 它们都是对utils::Arena的实例。

首先看HeapAllocatorArena, 它接收四个模板参数:

第一个参数HeapAllocator是分配器类型:

```
/* -----
 * HeapAllocator
 *
 * + uses malloc() for all allocations
 * + frees blocks with free()
 * -----
 */
class HeapAllocator {
public:
    HeapAllocator() noexcept = default;

    template <typename AREA>
    explicit HeapAllocator(const AREA&) { }

    // our allocator concept
    void* alloc(size_t size, size_t alignment = alignof(std::max_align_t), size_t extra = 0) {
        // this allocator doesn't support 'extra'
        assert(extra == 0);
        return aligned_alloc(size, alignment);
    }

    void free(void* p) noexcept {
        aligned_free(p);
    }

    void free(void* p, size_t) noexcept {
        this->free(p);
    }

    ~HeapAllocator() noexcept = default;

    void swap(HeapAllocator&) noexcept { }
};
```

其中, 核心就是alloc函数, 实际上调用了memalign.h中的aligned_alloc:

```
//memalign.h

inline void* aligned_alloc(size_t size, size_t align) noexcept {
    // 'align' must be a power of two and a multiple of sizeof(void*)
    align = (align < sizeof(void*)) ? sizeof(void*) : align;
    assert(align && !(align & align - 1));
    assert((align % sizeof(void*)) == 0);

    void* p = nullptr;

#ifdef WIN32
    p = ::_aligned_malloc(size, align);
#else
    ::posix_memalign(&p, align, size);
#endif
    return p;
}
```

这里使用POSIX接口posix_memalign来在堆中分配空间，size应该是align的倍数，而align应该是2的N次幂。

和HeapAllocator类似的还有LinearAllocator、PoolAllocator、STLAllocator。

第二个参数NoLock是加锁类型，定义在LockingPolicy命名空间内：

```
struct NoLock {
    void lock() noexcept { }
    void unlock() noexcept { }
};
```

可以看到，其中定义了两个空函数，一个是用来加锁的lock，一个是用来解锁的unlock。同样的还有SpinLock和Mutex。

第三个参数Untracked是跟踪类型，声明在TrackingPolicy命名空间下：

```
struct Untracked {
    Untracked() noexcept = default;
    Untracked(const char* name, void* base, size_t size) noexcept {
        (void)name, (void)base, (void)size;
    }
    void onAlloc(void* p, size_t size, size_t alignment, size_t extra) noexcept {
        (void)p, (void)size, (void)alignment, (void)extra;
    }
    void onFree(void* p, size_t = 0) noexcept { (void)p; }
    void onReset() noexcept { }
    void onRewind(void* addr) noexcept { (void)addr; }
};
```

可以看到，其中主要包括了四个成员函数：

onAlloc 分配内存时的回调

onFree 释放内存时的回调

onReset 复位内存时的回调

onRewind 释放内存到特定位置时的回调

这些回调发生在内存相关的不同时间点，我们这里不关注这些内存事件，所以Untracked内实现都为空，没有实际作用，类似的还有HighWatermark、Debug、DebugAndHighWatermark。

第四个参数NullArea是空的区域，定义在AreaPolicy命名空间中，代表了一段内存区域：

```
class NullArea {
public:
    void* data() const noexcept { return nullptr; }
    size_t size() const noexcept { return 0; }
};
```

这其中，也是定义了两个成员函数：

data 内存区域的数据

size 内存区域的大小

类似的还有StaticArea、HeapArea。

接着，我们看LinearAllocatorArena，它接收两个模板参数：

第一个参数LinearAllocator，如上面所说，是另外一种分配器，用来分配线性内存。

```
/* -----
 * LinearAllocator
 *
 * + Allocates blocks linearly
 * + Cannot free individual blocks
 * + Can free top of memory back up to a specified point
 * + Doesn't call destructors
 * -----
 */

class LinearAllocator {
public:
    // use memory area provided
    LinearAllocator(void* begin, void* end) noexcept;

    template <typename AREA>
    explicit LinearAllocator(const AREA& area) : LinearAllocator(area.begin(), area.end()) { }

    // Allocators can't be copied
    LinearAllocator(const LinearAllocator& rhs) = delete;
    LinearAllocator& operator=(const LinearAllocator& rhs) = delete;

    // Allocators can be moved
    LinearAllocator(LinearAllocator&& rhs) noexcept;
    LinearAllocator& operator=(LinearAllocator&& rhs) noexcept;

    ~LinearAllocator() noexcept = default;

    // our allocator concept
    void* alloc(size_t size, size_t alignment = alignof(std::max_align_t), size_t extra = 0) UTILS_RESTRICT {
        // branch-less allocation
        void* const p = pointermath::align(current(), alignment, extra);
        void* const c = pointermath::add(p, size);
        bool success = c <= end();
        set_current(success ? c : current());
        return success ? p : nullptr;
    }

    // API specific to this allocator

    void *getCurrent() UTILS_RESTRICT noexcept {
        return current();
    }

    // free memory back to the specified point
    void rewind(void* p) UTILS_RESTRICT noexcept {
        assert(p >= mBegin && p < end());
        set_current(p);
    }

    // frees all allocated blocks
    void reset() UTILS_RESTRICT noexcept {
        rewind(mBegin);
    }

    size_t allocated() const UTILS_RESTRICT noexcept {
        return mSize;
    }
}
```

```

size_t available() const UTILS_RESTRICT noexcept {
    return mSize - mCur;
}

void swap(LinearAllocator& rhs) noexcept;

void *base() noexcept { return mBegin; }

void free(void*, size_t) UTILS_RESTRICT noexcept { }

private:
void* end() UTILS_RESTRICT noexcept { return pointermath::add(mBegin, mSize); }
void* current() UTILS_RESTRICT noexcept { return pointermath::add(mBegin, mCur); }
void set_current(void* p) UTILS_RESTRICT noexcept {
    mCur = uint32_t(uintptr_t(p) - uintptr_t(mBegin));
}

void* mBegin = nullptr;
uint32_t mSize = 0;
uint32_t mCur = 0;
};

```

其中，最核心的依然是alloc成员函数，用来分配线性内存空间。

第二个参数NoLock，就是空的加解锁函数。

为什么没有其它参数？

我们看Arena声明：

```

// -----
// Arenas
// -----

template<typename AllocatorPolicy, typename LockingPolicy,
         typename TrackingPolicy = TrackingPolicy::Untracked,
         typename AreaPolicy = AreaPolicy::HeapArea>
class Arena {
public:

    Arena() = default;
    ...
};

```

可以看到，Arena的后两个模板参数TrackingPolicy和AreaPolicy，都是有默认参数的，其中一个Untracked，一个是HeapArea。相当于声明为：

```

using LinearAllocatorArena = utils::Arena<
    utils::LinearAllocator,
    utils::LockingPolicy::NoLock,
    utils::TrackingPolicy::Untracked,
    utils::AreaPolicy::HeapArea>;

```

第三个参数，依然是Untracked，表示我们不关心分配过程的各个时间点的事件。

最后的HeapArea，是我们的区域类型，这里是堆类型：

```

class HeapArea {
public:
    HeapArea() noexcept = default;

    explicit HeapArea(size_t size) {
        if (size) {
            // TODO: policy committing memory
            mBegin = malloc(size);
            mEnd = pointermath::add(mBegin, size);
        }
    }

    ~HeapArea() noexcept {
        // TODO: policy for returning memory to system
        free(mBegin);
    }

    HeapArea(const HeapArea& rhs) = delete;
    HeapArea& operator=(const HeapArea& rhs) = delete;
    HeapArea(HeapArea&& rhs) noexcept = delete;
    HeapArea& operator=(HeapArea&& rhs) noexcept = delete;

    void* data() const noexcept { return mBegin; }
    void* begin() const noexcept { return mBegin; }
    void* end() const noexcept { return mEnd; }
    size_t size() const noexcept { return uintptr_t(mEnd) - uintptr_t(mBegin); }

    friend void swap(HeapArea& lhs, HeapArea& rhs) noexcept {
        using std::swap;
        swap(lhs.mBegin, rhs.mBegin);
        swap(lhs.mEnd, rhs.mEnd);
    }

private:
    void* mBegin = nullptr;
    void* mEnd = nullptr;
};

```

这其中，最重要的是构造函数，HeapArea通过一个size参数来指定了区域大小，并通过malloc为其分配了内存空间。并将其起始地址保存到mBegin中，然后，通过pointermath::add方法来计算内存区域的结束位置，并保存到mEnd：

```

namespace pointermath {

template <typename P, typename T>
static inline P* add(P* a, T b) noexcept {
    return (P*)(uintptr_t(a) + uintptr_t(b));
}
}

```

可以看到，add传入两个模板参数，第一个，是指向一段内存的指针类型P，第二个是指针的偏移量T类型。add通过将指向P的内存地址a加上T类的偏移量b后，将最终的指针返回。

下面，就使用过程进行详细描述。

在FEngine的初始化列表中，将mPerRenderPassAllocator和mHeapAllocator初始化：

```

FEngine::FEngine(Backend backend, Platform* platform, const Config& config, void* sharedGLContext) :
    ...
    mPerRenderPassAllocator("FEngine::mPerRenderPassAllocator", config.perRenderPassArenaSizeMB * MiB),
    mHeapAllocator("FEngine::mHeapAllocator", AreaPolicy::NullArea{}),
    ...
{
    ...
}

```

最终分别调用到Arena的两个构造函数:

```
// -----  
// Arenas  
// -----  
  
template<typename AllocatorPolicy, typename LockingPolicy,  
        typename TrackingPolicy = TrackingPolicy::Untracked,  
        typename AreaPolicy = AreaPolicy::HeapArea>  
class Arena {  
public:  
  
    Arena() = default;  
  
    // construct an arena with a name and forward argument to its allocator  
    template<typename ... ARGS>  
    Arena(const char* name, size_t size, ARGS&& ... args)  
        : mArenaName(name),  
          mArea(size),  
          mAllocator(mArea, std::forward<ARGS>(args) ... ),  
          mListener(name, mArea.data(), mArea.size()) {  
  
    }  
  
    template<typename ... ARGS>  
    Arena(const char* name, AreaPolicy&& area, ARGS&& ... args)  
        : mArenaName(name),  
          mArea(std::forward<AreaPolicy>(area)),  
          mAllocator(mArea, std::forward<ARGS>(args) ... ),  
          mListener(name, mArea.data(), mArea.size()) {  
  
    }  
  
    ...  
};
```

一个传入name参数, 一个size参数。

一个传入name参数一个Area参数。

通过size参数和NullArea对象, 构造了内部的mArea对象, 并使用mArea对象构造内部的mAllocator对象, 并用name和mArea构造了传入的用来跟踪的TrackingPolicy类型mListener。

LinearAllocatorArena mPerRenderPassAllocator, 是HeapArea类型, 通过传入的size参数, 调用前面讲到的HeapArea带参构造函数创建mArea对象:

```
mArea(size)
```

HeapAllocatorArena mHeapAllocator的mArea, 是NullArea类型, 通过前面AreaPolicy::NullArea{} 创建的临时变量, 通过完美转发传入并调用拷贝构造函数创建mArea:

```
mArea(std::forward<AreaPolicy>(area))
```

接着, 将创建的mArea和完美转发的传入参数传递给mAllocator各自所属类型的构造函数来构造mAllocator:

```
class LinearAllocator {  
    explicit LinearAllocator(const AREA& area) : LinearAllocator(area.begin(), area.end()) { }  
};  
  
LinearAllocator::LinearAllocator(void* begin, void* end) noexcept  
    : mBegin(begin), mSize(reinterpret_cast<uintptr_t>(end) - reinterpret_cast<uintptr_t>(begin)) {  
}  
  
class HeapAllocator {  
    explicit HeapAllocator(const AREA&) { }  
};
```

可以看到, LinearAllocator使用了传入的area参数, 并将缓冲区起始地址和大小初始化, 而HeapAllocator并没有使用, 就是说它并不关心内部区域的信息。

当创建完两种Allocator对象之后，看看如何使用它创建我们实际对象的：

```
template<typename T>
inline T* FEngine::create(ResourceList<T>& list, typename T::Builder const& builder) noexcept {
    T* p = mHeapAllocator.make<T>(*this, builder);
    list.insert(p);
    return p;
}

FScene* FEngine::createScene() noexcept {
    FScene* p = mHeapAllocator.make<FScene>(*this);
    if (p) {
        mScenes.insert(p);
    }
    return p;
}

FView* FEngine::createView() noexcept {
    FView* p = mHeapAllocator.make<FView>(*this);
    if (p) {
        mViews.insert(p);
    }
    return p;
}

FFence* FEngine::createFence(FFence::Type type) noexcept {
    FFence* p = mHeapAllocator.make<FFence>(*this, type);
    if (p) {
        std::lock_guard guard(mFenceListLock);
        mFences.insert(p);
    }
    return p;
}
```

在代码中，我们只找到mHeapAllocator对象的使用过程，是通过HeapAllocatorArena的make方法类创建实际对象的。

可以看到，上面主要有两种使用方式，第一种，使用模板来创建了一个通用的函数模板来创建资源文件，另外一种，是直接使用mHeapAllocator对象创建我们需要的内存区域。

但不管哪种，都是使用的make函数来进行创建的，也就是Arena的成员函数make：

```
template<typename AllocatorPolicy, typename LockingPolicy,
        typename TrackingPolicy = TrackingPolicy::Untracked,
        typename AreaPolicy = AreaPolicy::HeapArea>
class Arena {
public:
    ...
    // allocate memory from arena with given size and alignment
    // (acceptable size/alignment may depend on the allocator provided)
    void* alloc(size_t size, size_t alignment = alignof(std::max_align_t), size_t extra = 0) noexcept {
        std::lock_guard<LockingPolicy> guard(mLock);
        void* p = mAllocator.alloc(size, alignment, extra);
        mListener.onAlloc(p, size, alignment, extra);
        return p;
    }

    // Allocate and construct an object
    template<typename T, size_t ALIGN = alignof(T), typename... ARGS>
    T* make(ARGS&& ... args) noexcept {
        void* const p = this->alloc(sizeof(T), ALIGN);
        return p ? new(p) T(std::forward<ARGS>(args)...) : nullptr;
    }
    ...
}
```

可以看到，make接收一个模板参数T，对应到上面就是FScene、FView和FFence，make中通过this来调用Arena自身的alloc方法，并通过mAllocator.alloc来最终调用到各种Allocator各自的alloc函数来分配空间，并使用placement new来将T分配到刚刚alloc的位置上。

其中，ALIGN指定了我们默认以T的对齐值对齐，并通过T的大小来分配空间，最终会将对齐的内容考虑进去。类型为对应到我们这里是HeapAllocator的alloc:

```
class HeapAllocator {
public:
    ...
    // our allocator concept
    void* alloc(size_t size, size_t alignment = alignof(std::max_align_t), size_t extra = 0) {
        // this allocator doesn't support 'extra'
        assert(extra == 0);
        return aligned_alloc(size, alignment);
    }
};
```

就是上面说过的，最终调用aligned_alloc来分配实际空间。